

Learning word embeddings with Gensim

In this part, we are going to perform word embeddings with Gensim - a very popular toolkit for Natural Language Processing (NLP). We will build two word embeddings: `word2vec` and `fastText`.

First, let's start to import some useful packages:

```
In [73]: # import Pandas, numpy, tensorflow, nltk, re, keras,datetime, gensim
import pandas as pd # For data manipulation and analysis
import numpy as np # For numerical computations
import tensorflow as tf # For machine learning and deep learning tasks
import nltk # for natural language processing tasks
import re # for regular expression operations
import keras # for building neural networks
import datetime # for working with dates and times
import gensim # for topic modeling and document similarity analysis
import gensim.models as word2vec # for word embeddings
from gensim.models import Word2Vec # for training word embeddings

import logging # for logging events and errors
# Enable inline plotting for matplotlib
%matplotlib inline
```

Data preparation

Dataset: "Bag of Words Meets Bags of Popcorn". We will work with not only the training set but also the unlabeled training set. Because we only want to learn word embedding (not building model) so we don't need labels in the set, making the unlabeled data useful.

Now, let's load them using `pandas` package.

```
In [75]: # Load labeled training data from a tab-separated file into a pandas DataFrame
train = pd.read_csv("labeledTrainData.tsv",
                    delimiter="\t", # Use tab character as the delimiter
                    header=0,       # Use the first row as the header
                    quoting=3)      # Ignore quoting in the file

# Load unlabeled training data from a tab-separated file into a pandas DataFrame
unlabeled = pd.read_csv("unlabeledTrainData.tsv",
                        delimiter="\t", # Use tab character as the delimiter
```

```
header=0,          # Use the first row as the header
quoting=3)         # Ignore quoting in the file
```

There are 25,000 reviews in the training set and 50,000 reviews in the unlabeled dataset.

```
In [77]: # Print the shape (number of rows and columns) of the labeled training dataset
print("Train set: {}".format(train.shape))

# Print the shape (number of rows and columns) of the unlabeled training dataset
print("Unlabeled set: {}".format(unlabeled.shape))
```

Train set: (25000, 3)

Unlabeled set: (50000, 2)

Let's take a quick review about the first unlabeled review. We can see that there are many HTML tags in this review. So our task is to clean up those reviews by removing the unwanted characters such as HTML tags and special characters (`\, /, ?, !, etc.`).

```
In [79]: # Access the second review in the unlabeled dataset
unlabeled.review[1]
```

```
Out[79]: '"I saw this film about 20 years ago and remember it as being particularly nasty. I believe it is based on a true incident: a young man breaks into a nurses\' home and rapes, tortures and kills various women.<br /><br />It is in black and white but saves the colour for one shocking shot.<br /><br />At the end the film seems to be trying to make some political statement but it just comes across as confused and obscene.<br /><br />Avoid.'"
```

Function `convert_to_sentences` is defined in order to remove special characters. Steps are as below:

1. Convert each review into sentences as we might have many sentences in one review by using `punkt` tokenizer from NLTK
2. Replace HTML tags with space. Character '-' with space also, and add space after '!' just to help tokenizer to strip sentence correctly. If we don't do this, sentences connected with '!' (no space after) will be considered as one sentence which is not correct. And replace double space with one space only.
3. Convert each sentence into words and apply the `special_characters` filter on it. Then convert all of the into lowercase.
4. Finally, return a list of reviews which contain inside each review a list of sentences made up a list of words.

```
In [81]: # Compile a regular expression pattern to match non-alphanumeric characters (except spaces)
special_characters = re.compile("[^A-Za-z0-9 ]")

# Load the NLTK Punkt tokenizer for English language
tokenizer = nltk.data.load("tokenizers/punkt/english.pickle")

# Define a function to convert text data into sentences and then into words
def convert_to_sentences(data, tokenizer):
    # Preprocess the text data: convert to lowercase and replace HTML tags and hyphens
    data = data.lower().replace("<br />", " ") # Remove HTML line breaks
    data = data.replace("-", " ") # Replace hyphens with spaces
```

```

data = data.replace(".", ". ") # Add space after periods

# Remove extra spaces (this line seems redundant, as it replaces space with space)
data = re.sub(" ", " ", data)

# Tokenize the text data into sentences using NLTK Punkt tokenizer
all_sentences = tokenizer.tokenize(data.strip())

# Initialize an empty list to store the processed sentences
sentences = []

# Iterate over each sentence and preprocess it
for words in all_sentences:
    # Remove special characters from the sentence and convert to lowercase
    s = re.sub(special_characters, "", words.lower())

    # Check if the sentence is not empty after preprocessing
    if len(s) > 0:
        # Split the sentence into words and append to the list of sentences
        sentences.append(s.split())

# Return the list of processed sentences
return sentences

```

Below is a sample of raw review:

```

In [83]: # Print the 91st review from the unlabeled dataset
print("Review: \n{}".format(unlabeled.review[90]))

```

Review:

"Steven Seagal returns in Black Dawn.Cool title.I cannot say the same for the movie though.It ranks as the Worst Seagal movie yet.Yes.It is worse than Out for a Kill.Did we really need a sequel to the Forigner that bad?

Steven r eturns as Jonathon Cold.Assassin for hire.This time he is out to stop a group of Terrorists and prevent Nuclier Bomb fr om hitting Los Angelas with the help of CIA agent Tamara Davies.

This idea seems okay.The Jonathon Cold char acter once again gives the big guy a chance to tarnish his clean-cut cinematic Image.You get the feeling with a decent director and production team this movie could have went somewhere.If this cost 15 million I wanna know where it was spe nt?It was not spent on the movie?!!

The action scenes are not bad.The movie was obviously shot like a Doug Liman flick.Its just that most of the Time Seagal is doubled so much you cannot take the movie seriously whatsoever.The script seems...what script?!Logic is Black Dawned for a dozen low-budget action movies.

Seagal is not too ba d here for a whole 15 minutes he is engaging then goes about the sleepwalking that he has done for his last 2 movies.Wh oever is doing his fighting is doing a commendable job.

The light that shines through is Tamara Davies as Co ld's sidekick.She has the chops,skills,looks and drive that give Black Dawn a little kick in the action department.How she got in this movie is a riddle.She should be a bigger star.Nicholas Davidoff and John Pyper Feron are not believab le bad guys.

Whose idea was it to make a sequel to a bad movie in the first place? It did not set the world on fire to my memory.I can only hope they inject the next Forigner sequel with some class,imagination,Less stunt double s and a livelier Seagal.If not then leave Jonathon Cold on Ice."

Applying the function `convert_to_sentences` to this review, we get 31 sentences in total.

```
In [85]: # Convert the 91st review into a list of sentences
sentences_sample = convert_to_sentences(unlabeled.review[90], tokenizer)

# Print the number of sentences in the formatted review
print("Length of formatted sentences: {}".format(len(sentences_sample)))
```

Length of formatted sentences: 31

Details of each sentence are as followed:

```
In [87]: # Print each sentence in the sample
for i in sentences_sample:
    print(i)
```

```

['steven', 'seagal', 'returns', 'in', 'black', 'dawn']
['cool', 'title']
['i', 'cannot', 'say', 'the', 'same', 'for', 'the', 'movie', 'though']
['it', 'ranks', 'as', 'the', 'worst', 'seagal', 'movie', 'yet']
['yes']
['it', 'is', 'worse', 'than', 'out', 'for', 'a', 'kill']
['did', 'we', 'really', 'need', 'a', 'sequel', 'to', 'the', 'foreigner', 'that', 'bad']
['steven', 'returns', 'as', 'jonathon', 'cold']
['assassin', 'for', 'hire']
['this', 'time', 'he', 'is', 'out', 'to', 'stop', 'a', 'group', 'of', 'terrorists', 'and', 'prevent', 'nuclier', 'bom
b', 'from', 'hitting', 'los', 'angelas', 'with', 'the', 'help', 'of', 'cia', 'agent', 'tamara', 'davies']
['this', 'idea', 'seems', 'okay']
['the', 'jonathon', 'cold', 'character', 'once', 'again', 'gives', 'the', 'big', 'guy', 'a', 'chance', 'to', 'tarnish',
'his', 'clean', 'cut', 'cinematic', 'image']
['you', 'get', 'the', 'feeling', 'with', 'a', 'decent', 'director', 'and', 'production', 'team', 'this', 'movie', 'coul
d', 'have', 'went', 'somewhere']
['if', 'this', 'cost', '15', 'million', 'i', 'wanna', 'know', 'where', 'it', 'was', 'spentit', 'was', 'not', 'spent',
'on', 'the', 'movie']
['the', 'action', 'scenes', 'are', 'not', 'bad']
['the', 'movie', 'was', 'obviously', 'shot', 'like', 'a', 'doug', 'liman', 'flick']
['its', 'just', 'that', 'most', 'of', 'the', 'time', 'seagal', 'is', 'doubled', 'so', 'much', 'you', 'cannot', 'take',
'the', 'movie', 'seriously', 'whatsoever']
['the', 'script', 'seems']
['what', 'script']
['logic', 'is', 'black', 'dawned', 'for', 'a', 'dozen', 'low', 'budget', 'action', 'movies']
['seagal', 'is', 'not', 'too', 'bad', 'here', 'for', 'a', 'whole', '15', 'minutes', 'he', 'is', 'engaging', 'then', 'go
es', 'about', 'the', 'sleepwalking', 'that', 'he', 'has', 'done', 'for', 'his', 'last', '2', 'movies']
['whoever', 'is', 'doing', 'his', 'fighting', 'is', 'doing', 'a', 'commendable', 'job']
['the', 'light', 'that', 'shines', 'through', 'is', 'tamara', 'davies', 'as', 'colds', 'sidekick']
['she', 'has', 'the', 'chopsskillslooks', 'and', 'drive', 'that', 'give', 'black', 'dawn', 'a', 'little', 'kick', 'in',
'the', 'action', 'department']
['how', 'she', 'got', 'in', 'this', 'movie', 'is', 'a', 'riddle']
['she', 'should', 'be', 'a', 'bigger', 'star']
['nicholas', 'davidoff', 'and', 'john', 'pyper', 'feruson', 'are', 'not', 'believable', 'bad', 'guys']
['whose', 'idea', 'was', 'it', 'to', 'make', 'a', 'sequel', 'to', 'a', 'bad', 'movie', 'in', 'the', 'first', 'place']
['it', 'did', 'not', 'set', 'the', 'world', 'on', 'fire', 'to', 'my', 'memory']
['i', 'can', 'only', 'hope', 'they', 'inject', 'the', 'next', 'foreigner', 'sequel', 'with', 'some', 'classimaginationle
ss', 'stunt', 'doubles', 'and', 'a', 'livelier', 'seagal']
['if', 'not', 'then', 'leave', 'jonathon', 'cold', 'on', 'ice']

```

Now, let's do the same for all of our train and unlabeled reviews. Here we will join 2 lists into 1. This may take several minutes to complete due to large number of reviews needed to be clean up (75,000).

```

In [89]: # Initialize an empty list to store tokenized sentences
sentences = []

# Process the training set reviews

```

```

for review in train.review:
    # Convert each review to sentences and add to the list
    sentences += convert_to_sentences(review, tokenizer)
print("Done for train set.")

# Process the unlabeled set reviews
for review in unlabeled.review:
    # Convert each review to sentences and add to the list
    sentences += convert_to_sentences(review, tokenizer)
print("Done for unlabeled set.")

```

Done for train set.

Done for unlabeled set.

Let's check first 5 sentences:

```

In [91]: # Print the first 5 sentences
for i in sentences[0:5]:
    print("{}\n".format(i))

```

```

['with', 'all', 'this', 'stuff', 'going', 'down', 'at', 'the', 'moment', 'with', 'mj', 'ive', 'started', 'listening',
'to', 'his', 'music', 'watching', 'the', 'odd', 'documentary', 'here', 'and', 'there', 'watched', 'the', 'wiz', 'and',
'watched', 'moonwalker', 'again']

```

```

['maybe', 'i', 'just', 'want', 'to', 'get', 'a', 'certain', 'insight', 'into', 'this', 'guy', 'who', 'i', 'thought', 'w
as', 'really', 'cool', 'in', 'the', 'eighties', 'just', 'to', 'maybe', 'make', 'up', 'my', 'mind', 'whether', 'he', 'i
s', 'guilty', 'or', 'innocent']

```

```

['moonwalker', 'is', 'part', 'biography', 'part', 'feature', 'film', 'which', 'i', 'remember', 'going', 'to', 'see', 'a
t', 'the', 'cinema', 'when', 'it', 'was', 'originally', 'released']

```

```

['some', 'of', 'it', 'has', 'subtle', 'messages', 'about', 'mjs', 'feeling', 'towards', 'the', 'press', 'and', 'also',
'the', 'obvious', 'message', 'of', 'drugs', 'are', 'bad', 'mkay']

```

```

['visually', 'impressive', 'but', 'of', 'course', 'this', 'is', 'all', 'about', 'michael', 'jackson', 'so', 'unless',
'you', 'remotely', 'like', 'mj', 'in', 'anyway', 'then', 'you', 'are', 'going', 'to', 'hate', 'this', 'and', 'find', 'i
t', 'boring']

```

word2vec embedding

Before doing the embedding, let's define values for the model.

- `num_feature` : The dimension of word vector. The more dimension the better representation but this is going to take more time to learn and more data. However, since we don't have that much data, let's set this to 50 only (default is 100).